

SAP-Assignment 1

Shipping on the Air

Marco Morelli

1. ANALISI

1.1 Descrizione del problema

Il sistema realizza un servizio online di consegna di pacchi tramite droni. Il bisogno principale è il seguente: un utente intende spedire un pacco, di un determinato peso, da un luogo a un altro, in una data e ora specifiche, eventualmente anche immediatamente, entro un determinato lasso di tempo. L'utente può inoltre tracciare la consegna, conoscendone la posizione e il tempo rimanente al completamento.

Da tale bisogno derivano i requisiti funzionali del sistema. L'utente deve potersi registrare ed effettuare il login prima di accedere al servizio. Una volta autenticato come Sender, può creare una richiesta di consegna specificando il peso del pacco, il luogo di partenza e quello di destinazione, scegliendo se la consegna debba essere immediata oppure programmata a una data e ora precise, e indicando una deadline massima entro cui la consegna deve completarsi. Durante l'esecuzione, il Sender può tracciare la consegna in tempo reale, ricevendo la posizione aggiornata del pacco e il tempo stimato rimanente.

Oltre al nucleo richiesto dalla traccia, il prototipo include alcune estensioni che arricchiscono il modello di dominio: la cancellazione di una consegna da parte del Sender prima del volo, una validazione esplicita della richiesta con feedback immediato di accettazione o rifiuto, l'assegnazione automatica del drone idoneo più vicino per le consegne immediate, una persona Admin che monitora lo stato della flotta e osserva la pianificazione delle consegne, e la gestione del fallimento di un drone in volo, nell'intento di riprodurre un sistema semi-realistico. L'Admin è deliberatamente un attore di sola osservazione: non interviene sul ciclo di vita delle consegne né sulla pianificazione, gestiti automaticamente dal sistema, ma fruisce di viste di sola lettura su flotta e scheduling. In un sistema di gestione di una flotta il monitoraggio costituisce infatti un'esigenza distinta dall'operatività; modellarlo come attore separato consente di esibire la segregazione dei ruoli (un Sender non vede la flotta, un Admin non crea consegne) e di fornire un consumatore concreto ai read model del sistema.

1.2 Processo di analisi

Il primo passo è stata l'analisi dei requisiti del sistema tramite i **casi d'uso**, raccolti e rappresentati attraverso il diagramma UML in figura. Il diagramma distingue i due attori umani del sistema, il Sender e l'Admin, e mostra come le azioni

operative (creare, tracciare e cancellare una consegna, monitorare la flotta, osservare la pianificazione) richiedano sempre l'autenticazione preliminare.

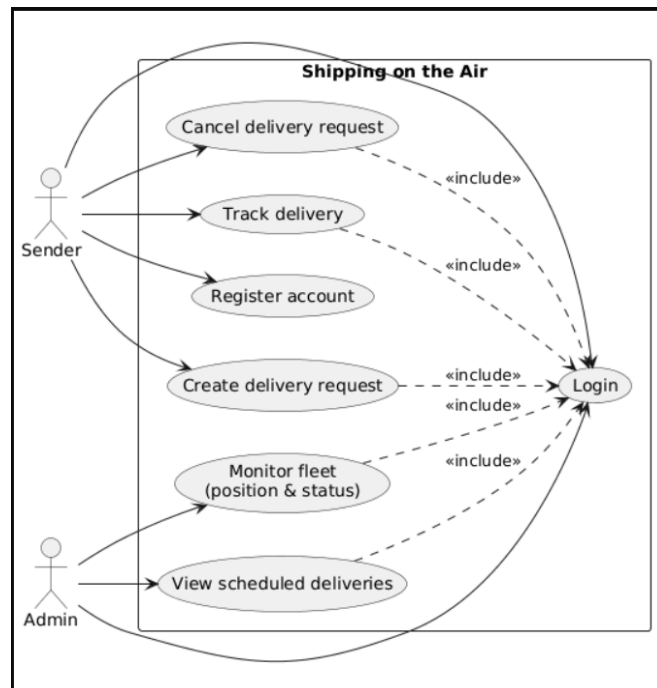


Diagramma dei casi d'uso

A partire dai casi d'uso vengono individuate le **User Stories**, che descrivono il valore atteso dal punto di vista di chi utilizza il sistema. Se ne riportano alcuni esempi:

As a new user,
I want to register an account on the delivery service
so that I can request drone deliveries.

As a logged-in Sender,
I want to create a delivery request specifying pickup location and destination
so that my package can be shipped from one place to another.

As a logged-in Sender,
I want to choose between immediate or scheduled delivery
so that my package is picked up either now or at a date/time I decide.

As a logged-in Sender,
I want to track my delivery in real time
so that I know the current position of my package and the estimated time remaining.

As an Admin,
I want to monitor the position and status of every drone in real time
so that I have an overview of the fleet.

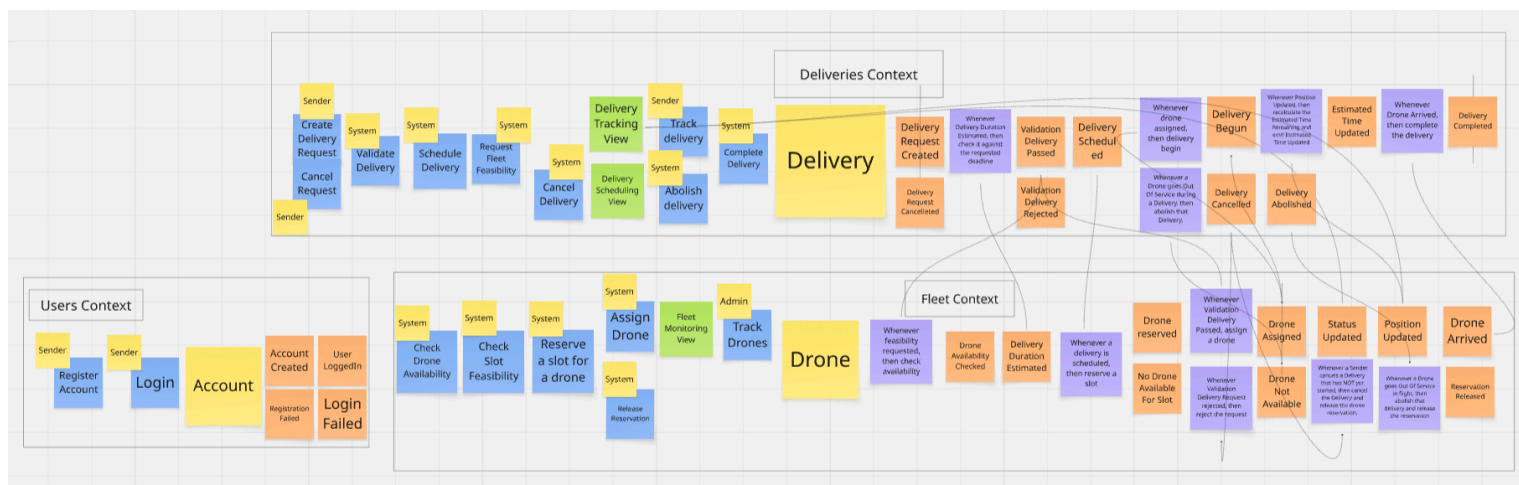
Da ciascuna User Story sono stati derivati i relativi **Acceptance Criteria** sotto forma di scenari. Se ne riporta un esempio:

```
Scenario: Successful registration
  Given I am on the registration page
  When I register with username "user-1" and password "Secret#123"
  Then I should see a confirmation that my account has been created
  And I should be able to log in
```

Adottando uno stile **Behavior-Driven Development (BDD)**, sono stati raccolti tutti gli scenari ed impiegati per realizzare gli **Acceptance Test** automatici tramite il tool **Cucumber**, così da legare direttamente i requisiti espressi in linguaggio naturale al comportamento verificabile del sistema.

1.3 Domain-Driven Design (DDD)

Come approccio metodologico viene adottato il **Domain-Driven Design**, applicandolo dall'analisi fino allo sviluppo. Il primo passo è stata la realizzazione dell'Event Storming per visualizzare il flusso degli eventi di dominio e i comportamenti attesi. Questo diagramma dispone in sequenza comandi, eventi di dominio e policy lungo l'intero ciclo di vita di una consegna: dalla richiesta alla validazione, all'assegnazione del drone, fino al completamento, includendo i rami di rifiuto, cancellazione e terminazione forzata.



Event Storming del dominio

Il passo successivo è consistito nella definizione dell'**Ubiquitous Language**, il vocabolario condiviso con cui descrivere il dominio in modo univoco. Se ne riportano alcuni termini centrali:

- **Delivery:** l'aggregato che rappresenta il trasporto di un pacco dal mittente a una destinazione, dotato di un proprio ciclo di vita dalla richiesta al completamento.
- **Delivery status:** lo stato corrente della consegna lungo il suo ciclo di vita (esempio: *SCHEDULED*, *IN_PROGRESS*, *DELIVERED*).

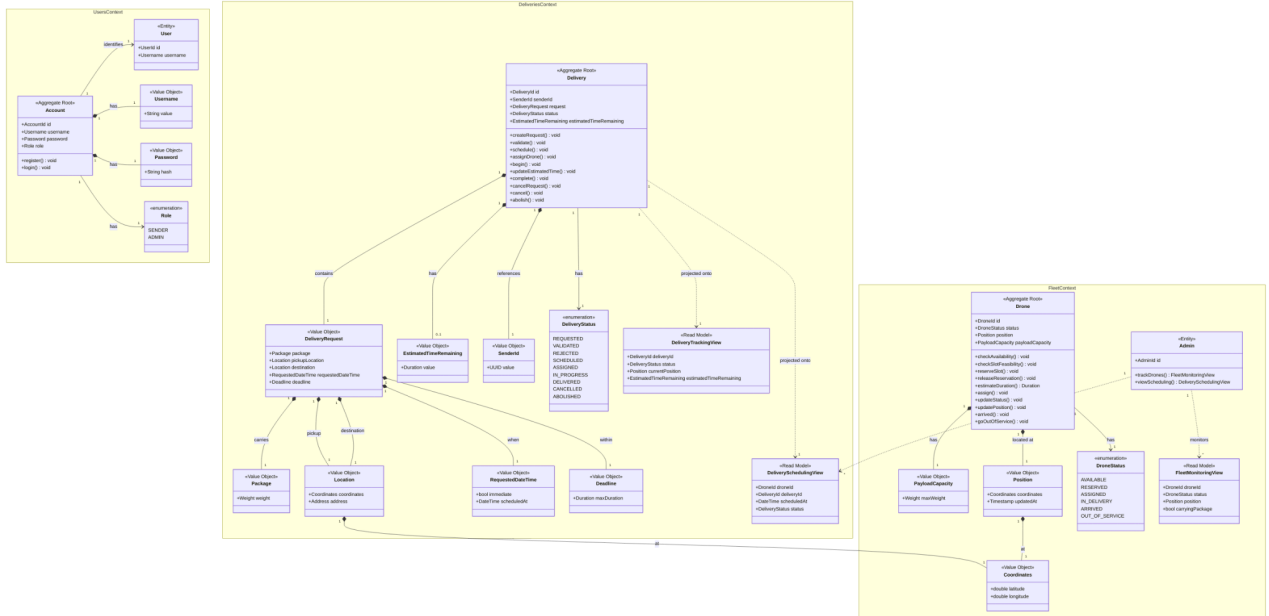
- **Estimated Time Remaining (ETR):** il tempo stimato rimanente al completamento, ricalcolato a ogni aggiornamento di posizione e azzerato all'arrivo.
- **Location:** l'indirizzo, composto da coordinate geografiche e da un indirizzo testuale, utilizzato come punto di partenza e di destinazione di una consegna.
- **Position:** il punto in cui un drone si trova in un dato istante, composto da coordinate e dall'istante di osservazione.
- **Drone:** l'entità della flotta che esegue fisicamente la spedizione, caratterizzata da uno stato operativo, una posizione e una capacità di carico.
- **Requested date/time:** la data e l'ora specificate dall'utente per la spedizione, oppure l'indicazione di consegna immediata.

Successivamente è stato modellato il dominio identificando i concetti e collegandoli ai relativi **Building Blocks del DDD** (entity, value object, aggregate, domain event, repository). Nel codice tale legame è reso esplicito facendo estendere alle interfacce che rappresentano i concetti del dominio le interfacce dei building block, raccolte in un modulo common, così che la natura di ciascun concetto sia dichiarata e verificabile.

In fase di modellazione sono stati identificati tre **Bounded Context**:

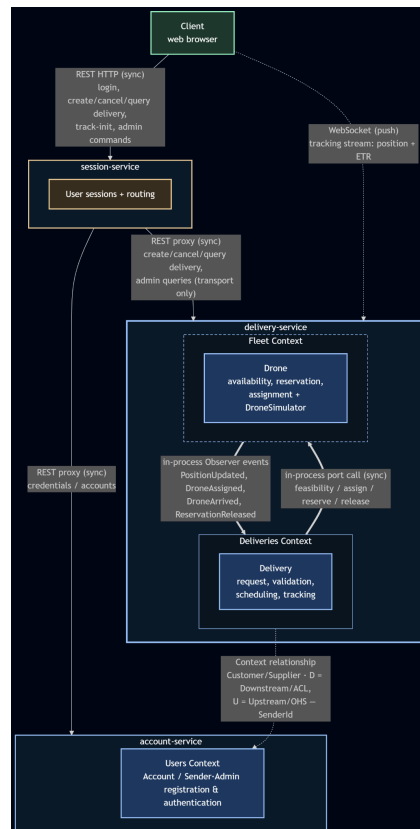
- *Users Context:* racchiude i concetti legati agli account e all'autenticazione. L'aggregato centrale è Account, che porta un attributo role (SENDER o ADMIN); Sender e Admin costituiscono due proiezioni dello stesso aggregato, distinte dal ruolo.
- *Deliveries Context:* racchiude la consegna come aggregato, dalla creazione e validazione fino al tracciamento e al completamento.
- *Fleet Context:* racchiude i droni, la loro disponibilità, prenotazione, assegnazione e la telemetria interna.

Il modello di dominio risultante, con i tre context e le relazioni tra i loro concetti, è riportato in figura.



Modello di Dominio

Un punto rilevante emerso in fase di analisi riguarda la relazione tra i context. L'unica vera relazione di context-mapping del sistema è quella tra Deliveries e Users: il Deliveries Context dipende dall'identità conosciuta in Users, di cui utilizza il *SenderId*, ponendosi come Downstream/Customer con un livello anti-corruzione, mentre Users agisce da Upstream/Supplier. Il context map che sintetizza tali relazioni e i canali di comunicazione è mostrato sotto.



Context map

1.4 Requisiti non funzionali

In fase di analisi sono stati individuati i requisiti non funzionali che il sistema deve soddisfare e che, come tali, hanno vincolato a monte le scelte di progettazione. Se ne riportano i principali:

- **Sicurezza e controllo degli accessi:** ogni azione operativa deve essere subordinata all'autenticazione dell'utente, e i privilegi amministrativi non devono essere ottenibili tramite la registrazione pubblica.
- **Latenza e reattività del tracciamento:** gli aggiornamenti di posizione e di tempo rimanente devono essere recapitati all'utente che traccia con bassa latenza e in modo continuo.
- **Manutenibilità ed evolvibilità:** il dominio deve essere isolato dalla tecnologia, così che il sistema sia semplice da modificare ed estendere e che il rispetto dello stile architetturale sia verificabile.
- **Disponibilità ed efficienza sotto carico:** il sistema deve restare reattivo anche in presenza di molte consegne concorrenti e di numerose connessioni di tracciamento aperte, senza che le operazioni di lunga durata compromettano il servizio delle richieste brevi.
- **Affidabilità e robustezza:** il sistema deve reagire in modo controllato e prevedibile alle condizioni anomale (richieste non valide, fallimento di un drone in volo, dati che violano le invarianti di dominio).
- **Persistenza:** i dati che devono sopravvivere al riavvio del sistema devono essere resi durevoli, mentre gli stati intrinsecamente temporanei non richiedono persistenza.

2. DESIGN E ARCHITETTURA

Lo stile architetturale adottato è stato quello a microservizi e, per ciascun microservizio, è stata applicata l'architettura esagonale. Sono stati realizzati tre microservizi: account service, session service e delivery service. Ciascuno espone un'interfaccia **REST API**, documentata tramite OpenAPI, attraverso cui interagire con il sistema.

account Account registration, retrieval and authentication			^
POST	/accounts	Register a new Sender account	▼
GET	/accounts/{accountId}	Get account info	▼
POST	/accounts/login	Authenticate an account by username (Sender or Admin)	▼

session Login and user-session lifecycle		^
POST	/login Log in and open a user session	▼
delivery Sender delivery commands, routed to the delivery-service		^
POST	/user-sessions/{sessionId}/create-delivery Create a delivery request	▼
POST	/user-sessions/{sessionId}/cancel-delivery Cancel a delivery before flight	▼
POST	/user-sessions/{sessionId}/track-delivery Start tracking a delivery	▼
GET	/user-sessions/{sessionId}/deliveries/{deliveryId} Query a delivery's current state	▼
admin Admin read-only views, routed to the delivery-service		^
GET	/user-sessions/{sessionId}/admin/fleet View the fleet monitoring view	▼
GET	/user-sessions/{sessionId}/admin/scheduling View scheduled deliveries	▼
delivery Delivery lifecycle		^
POST	/deliveries Create and validate a delivery request	▼
GET	/deliveries/{deliveryId} Get the current observable state of a delivery	▼
POST	/deliveries/{deliveryId}/cancel Cancel a delivery before flight	▼
tracking Tracking command		^
POST	/deliveries/{deliveryId}/track Start tracking	▼
GET	/track/{trackingSessionId} Live tracking stream	▼
admin-fleet Fleet monitoring view		^
GET	/admin/fleet Fleet monitoring view	▼
admin-scheduling Delivery scheduling view		^
GET	/admin/scheduling Delivery scheduling view	▼

Visualizzazione tramite Swagger della specifica OpenAPI

La scelta dei tre microservizi segue i confini del dominio. Il Users Context, dotato di dati e responsabilità proprie e nettamente separato dalle consegne, diventa l'account service. Il Deliveries Context, cuore di dominio del sistema, diventa il delivery service. Il session service nasce come punto d'ingresso unico per il client in modo che orchestri il login e instradi i comandi verso gli altri servizi.

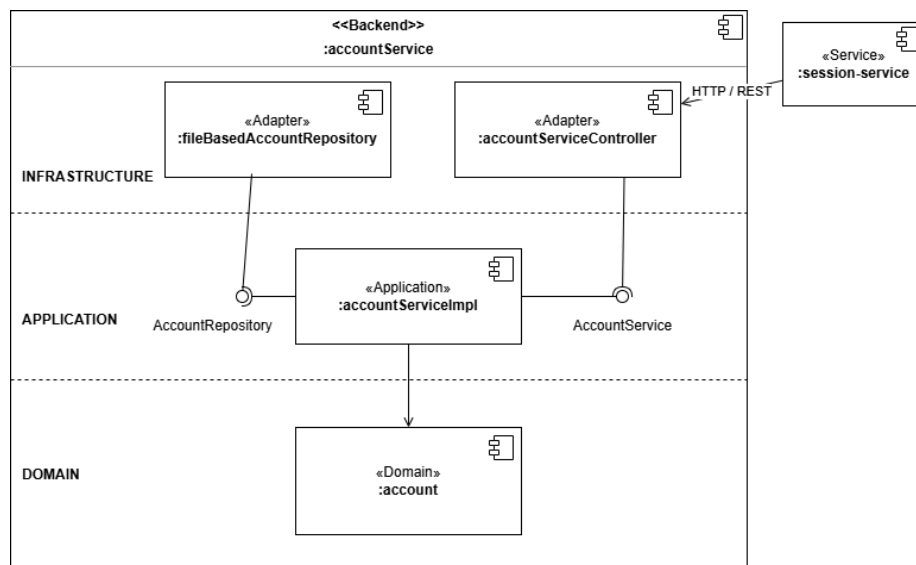
Una decisione architetturale rilevante riguarda il Fleet Context. Pur essendo un bounded context logico a sé stante, esso non costituisce un confine di deployment: è realizzato come modulo interno al delivery service. La ragione risiede nel fatto che Deliveries e Fleet collaborano in modo stretto e sincrono lungo il cammino critico di validazione, verifica di fattibilità, assegnazione del

drone, avvio della consegna; separarli come servizi distinti introdurrebbe chiamate di rete su tale cammino e, in assenza di message broker, costringerebbe a simulare uno stream di eventi cross-servizio. Mantenendoli nello stesso processo, con un confine logico netto ma un deployment condiviso, si ottiene la separazione concettuale tipica del DDD senza il costo della comunicazione distribuita.

2.1 Account service

L'account service si occupa della gestione degli account, permettendone la registrazione e l'autenticazione e garantendone la persistenza tramite repository su file. È l'unico proprietario dei dati delle credenziali e non ha conoscenza né delle consegne né dei droni. L'aggregato *Account* porta un attributo *role*: la registrazione pubblica crea esclusivamente account con ruolo **SENDER**, mentre l'account **Admin** è precaricato all'avvio del servizio e può unicamente effettuare il login.

L'architettura del microservizio segue quella mostrata in figura, che evidenzia l'utilizzo di *AccountService* come inbound port, implementata dall'adapter *AccountServiceController*, e di *AccountRepository* come outbound port, implementata a sua volta dall'adapter *FileBasedAccountRepository*.



C&C diagram dell'account service

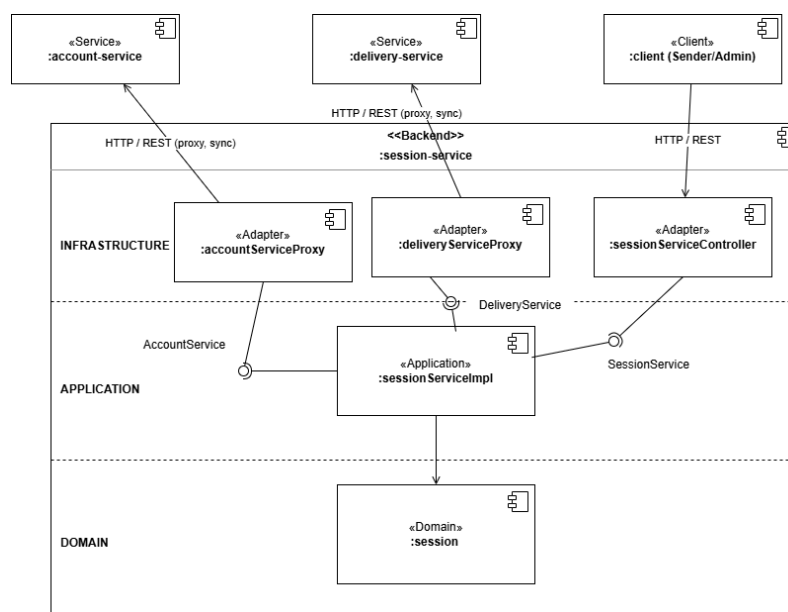
2.2 Session service

Il session service si occupa della gestione delle sessioni utente e dell'orchestrazione delle azioni: il login (in comunicazione con l'*account service*), la creazione, la cancellazione, il tracciamento e l'interrogazione delle consegne (in comunicazione con il *delivery service*), nonché le query di sola lettura dell'*Admin* verso la flotta e la pianificazione. Costituisce il punto d'ingresso unico per il client, il quale non comunica mai direttamente con gli altri servizi.

La comunicazione con gli altri microservizi avviene tramite proxy, che effettuano richieste HTTP basate sulle REST API esposte. Tale comunicazione è gestita in maniera sincrona, pur nella consapevolezza che ciò comporti il rischio di bloccare un event-loop di *Vert.x* in attesa di una risposta. Per questa ragione, nel controller le invocazioni che scatenano una chiamata proxy vengono eseguite al di fuori dell'event-loop, evitando il blocco. Le sessioni utente non sono mantenute in modo persistente, essendo temporanee.

È opportuno sottolineare che il session service è un orchestratore sottile che non contiene alcuna logica di dominio di consegna o di flotta, la sua unica responsabilità di dominio è la gestione delle sessioni e, per il resto, si limita a instradare. L'adozione di un orchestratore introduce, sul piano teorico, un accoppiamento maggiore rispetto a una pura coreografia; tale costo è accettato consapevolmente in cambio del punto d'ingresso unico imposto dall'assenza di gateway, ed è mitigato dal fatto che lo stream persistente di tracciamento non transita attraverso l'orchestratore, come descritto più avanti.

L'architettura del microservizio segue quella mostrata sotto, evidenzia l'utilizzo di *SessionService* come inbound port, mentre *AccountService* e *DeliveryService* fungono da outbound port, implementate dagli adapter *AccountServiceProxy* e *DeliveryServiceProxy*.



C&C diagram del session service

2.3 Delivery service (Deliveries + Fleet)

Il delivery service ha il compito di gestire l'intero ciclo di vita delle consegne, creazione, validazione, scheduling, esecuzione, tracciamento, cancellazione e terminazione forzata, garantendone la persistenza tramite repository su file, e di gestire la flotta di droni attraverso il modulo *Fleet* interno. Oltre alla REST API delle consegne, il servizio espone un endpoint amministrativo dedicato, attraverso cui

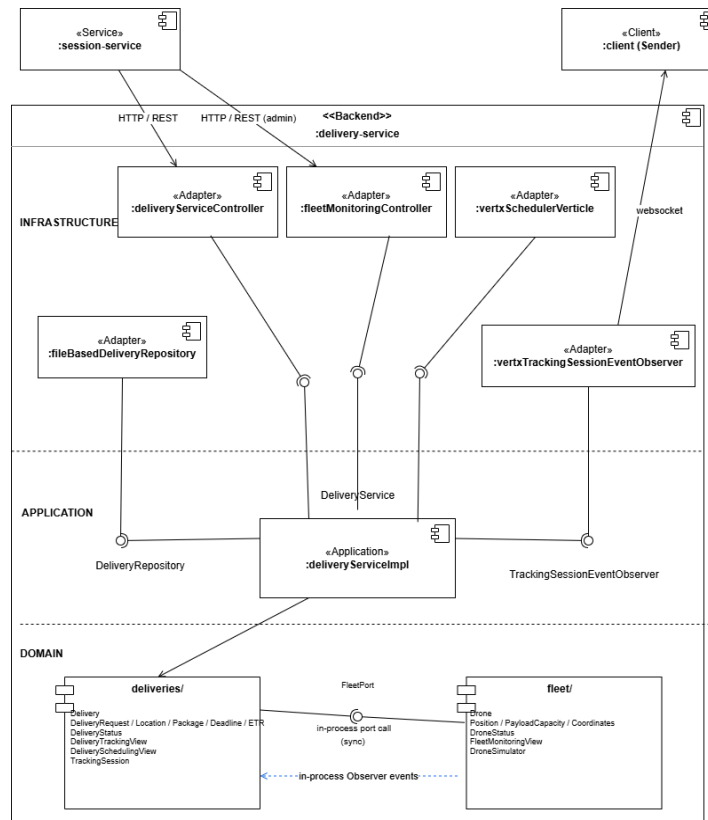
l'Admin osserva in sola lettura lo stato della flotta e la pianificazione delle consegne.

Internamente, i droni sono simulati come entità autonome: ogni drone attivo in una consegna è un **Virtual Thread** che avanza verso la destinazione e genera eventi per notificare gli aggiornamenti del processo. Il servizio modella il proprio comportamento tramite un'architettura ad eventi, facendo leva sul pattern **Observer**. I domain event prodotti dal movimento del drone *PositionUpdated* e *DroneArrived* sono propagati in-process: il modulo *Fleet* li notifica al *Deliveries Context*, che ricalcola l'*ETR* ed emette a sua volta *EstimatedTimeUpdated*, e tali aggiornamenti vengono inoltrati all'utente qualora stia tracciando la consegna. Per modellare il fatto che un utente stia tracciando una delivery, il servizio utilizza delle *TrackingSession*; analogamente alle sessioni utente, neppure le sessioni di tracciamento sono persistenti.

La comunicazione in tempo reale verso l'utente, a livello di infrastruttura, è realizzata con **Vert.x** e le **WebSocket**, che creano un canale di push per la posizione e il tempo rimanente. È significativo che tale stream sia stabilito direttamente tra client e delivery service, e non attraverso il session service: trattandosi di un canale persistente e di lunga durata, farlo transitare per l'orchestratore avrebbe comportato il mantenimento di una connessione aperta per ogni utente che traccia, con conseguente saturazione degli event-loop. In questo modo, invece, l'orchestratore resta esposto al rischio di blocco soltanto sulle chiamate brevi, e mai sul canale lungo.

Le consegne programmate sono gestite in modo automatico da uno *Scheduler*, un componente a timer che, allo scoccare dello slot richiesto, avvia l'assegnazione del drone precedentemente prenotato. Dal punto di vista dell'architettura esagonale, lo Scheduler è un driving adapter collocato nell'infrastruttura, del tutto analogo al controller REST: dove il controller traduce una richiesta HTTP in una chiamata alla porta del servizio, lo Scheduler traduce un evento temporale nella medesima chiamata. L'Admin non interviene sulla pianificazione, ma si limita a osservarla.

L'architettura del microservizio segue quella mostrata successivamente, che evidenzia l'utilizzo di *DeliveryService* come inbound port, mentre *DeliveryRepository* e *TrackingSessionEventObserver* fungono da outbound port, implementate dagli adapter *FileBasedDeliveryRepository* e *VertxTrackingSessionEventObserver*. All'interno del servizio i moduli deliveries e fleet collaborano tramite chiamate a porte interne e notifiche *Observer*, sempre in-process.



C&C diagram del delivery service (Deliveries + Fleet)

2.4 Comunicazione tra i servizi

I canali di comunicazione del sistema riflettono le scelte appena descritte. Il client comunica con il session service via **REST** sincro per tutti i comandi (login, creazione, cancellazione e interrogazione di consegne, avvio del tracciamento, query dell'Admin). Il session service comunica a sua volta con l'account service e con il delivery service via proxy **REST** sincro. L'unico canale di push verso il client è la **WebSocket** di tracciamento, aperta direttamente verso il delivery service. All'interno del delivery service, infine, i moduli Deliveries e Fleet, insieme al Virtual Thread del drone, collaborano in-process tramite chiamate dirette e pattern Observer, senza alcun passaggio di rete.

2.5 Risposta architetturale ai requisiti non funzionali

Le scelte architetturali descritte nelle sezioni precedenti rispondono in modo diretto ai requisiti non funzionali individuati in fase di analisi.

- **Sicurezza e controllo degli accessi:** ogni azione operativa è subordinata all'autenticazione. La registrazione pubblica produce esclusivamente account con ruolo SENDER, mentre l'account Admin è precaricato e non promuovibile, così che i privilegi amministrativi non siano ottenibili per via pubblica.
- **Latenza e reattività del tracciamento:** il requisito di bassa latenza è soddisfatto dal canale WebSocket di push stabilito direttamente tra client e

delivery service, senza passare per l'orchestratore, e dall'uso dei read model che servono le query senza caricare il modello di scrittura degli aggregati.

- **Manutenibilità ed evolvibilità:** l'architettura esagonale isola il dominio dalla tecnologia, mentre i confini dei bounded context e la chiara direzione delle dipendenze rendono il sistema più semplice da modificare ed estendere.
- **Disponibilità ed efficienza sotto carico:** la natura non bloccante di Vert.x è preservata eseguendo le chiamate proxy sincrone al di fuori dell'event-loop e mantenendo i canali di lunga durata separati dall'orchestratore, così da evitarne la saturazione. La simulazione dei droni tramite Virtual Thread consente di gestire molte consegne concorrenti con un costo per-thread contenuto.
- **Affidabilità e robustezza:** il sistema reagisce in modo controllato alle condizioni anomale. Una richiesta non valida è rifiutata con un evento unico di rifiuto, indipendentemente dalla condizione fallita; il fallimento di un drone in volo determina l'abolizione della relativa consegna e la liberazione della prenotazione.
- **Persistenza:** i dati che devono sopravvivere al riavvio (account e consegne) sono resi persistenti tramite repository su file, mentre gli stati intrinsecamente temporanei (sessioni utente e sessioni di tracciamento) sono mantenuti solo in memoria.

3. TESTING

La strategia di test combina due livelli complementari, scelti per fornire garanzie sia sul comportamento osservabile del sistema sia sul rispetto dello stile architetturale.

Da un lato ho realizzato gli **Acceptance Test** a partire dagli scenari **BDD**, utilizzando il tool **Cucumber**. Gli scenari, redatti in **Gherkin**, sono collegati a step definition in Java ed eseguiti da un runner che li raccoglie automaticamente. Vengono coperte le user story relative alla registrazione e al login, alla creazione di consegne immediate e programmate, alla cancellazione, al tracciamento, al monitoraggio della flotta, alla pianificazione, all'assegnazione del drone più vicino e alla difesa delle invarianti del dominio della flotta; per ciascuna sono previsti sia scenari di successo sia di fallimento, ove rilevante.

Dall'altro lato sono stati realizzati dei test architetturali, con cui si verifica automaticamente che ogni microservizio rispetti lo stile esagonale. Tali verifiche controllano che il dominio non dipenda dal layer applicativo o dall'infrastruttura, che le dipendenze tra i layer rispettino la direzione prevista, orientata verso il dominio, e che porte e adapter risiedano nei layer corretti.

4. CONCLUSIONI

L'adozione del **Domain-Driven Design** ha apportato diversi vantaggi, in particolare nella fase di analisi e raccolta dei requisiti, consentendo di concentrarsi inizialmente sulla business logic, astraendo dagli aspetti tecnologici e implementativi. La definizione dell'Ubiquitous Language, dell'Event Storming e del modello di dominio ha reso espliciti i concetti, gli eventi e le regole del sistema prima ancora della scrittura del codice; la creazione delle user stories e della specifica OpenAPI ha inoltre favorito l'individuazione delle risorse, delle azioni e dei comportamenti attesi, fornendo al contempo una base solida per i test.

L'architettura a microservizi ha consentito di focalizzarsi su un singolo bounded context alla volta, facilitandone il design e la conseguente implementazione. L'architettura esagonale adottata per i singoli microservizi ha permesso di separare con chiarezza gli aspetti di dominio da quelli tecnologici e implementativi, evitando che il dominio dipenda da questi ultimi: una proprietà che, oltre a essere documentata, è verificata dai test architetturali e che rende il sistema pronto ad evolvere.